
Курс: Рекомендательные системы

Занятие 14: RL in RecSys

Подготовил: Лашинин Олег

Дата: 22.04.25

План занятия



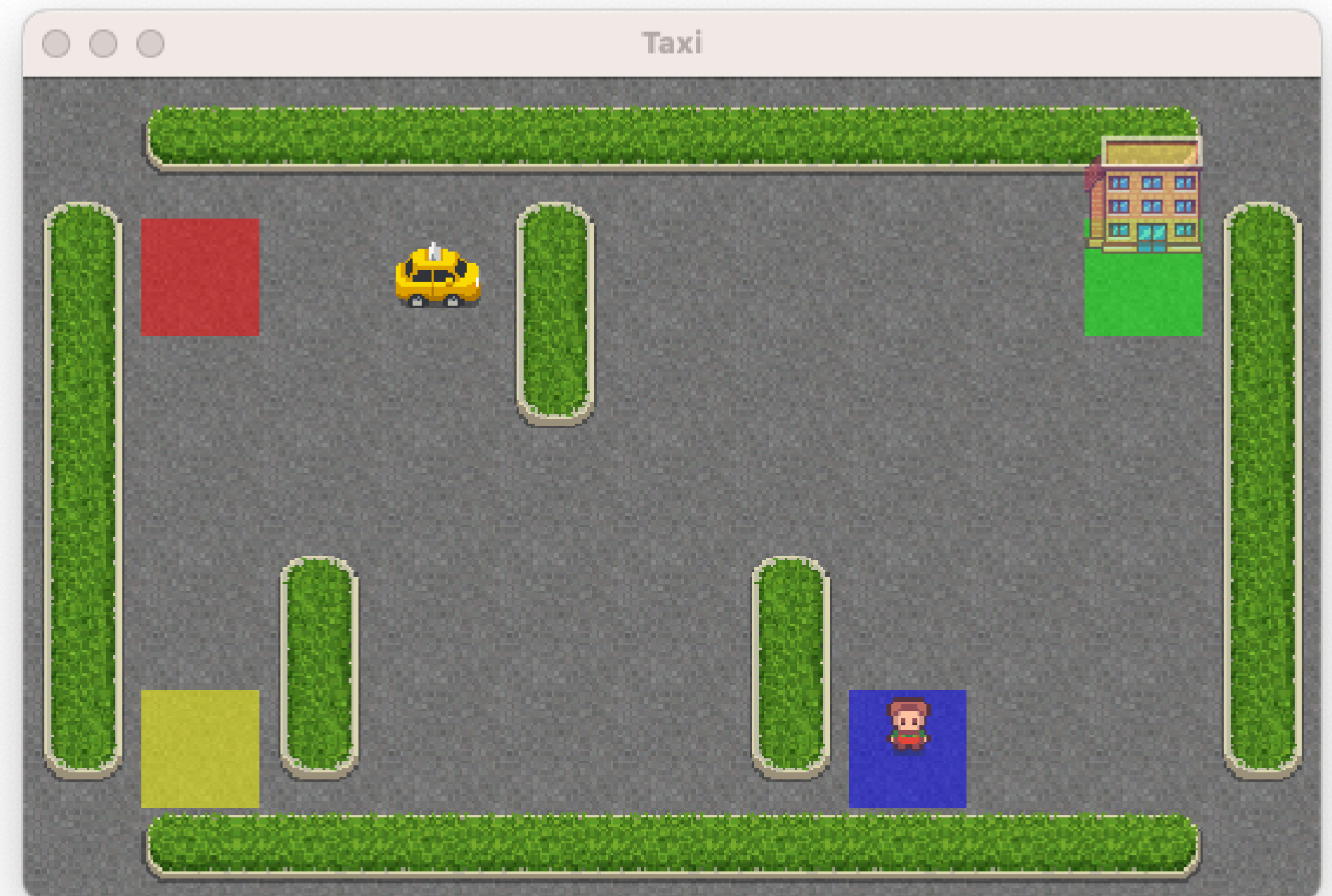
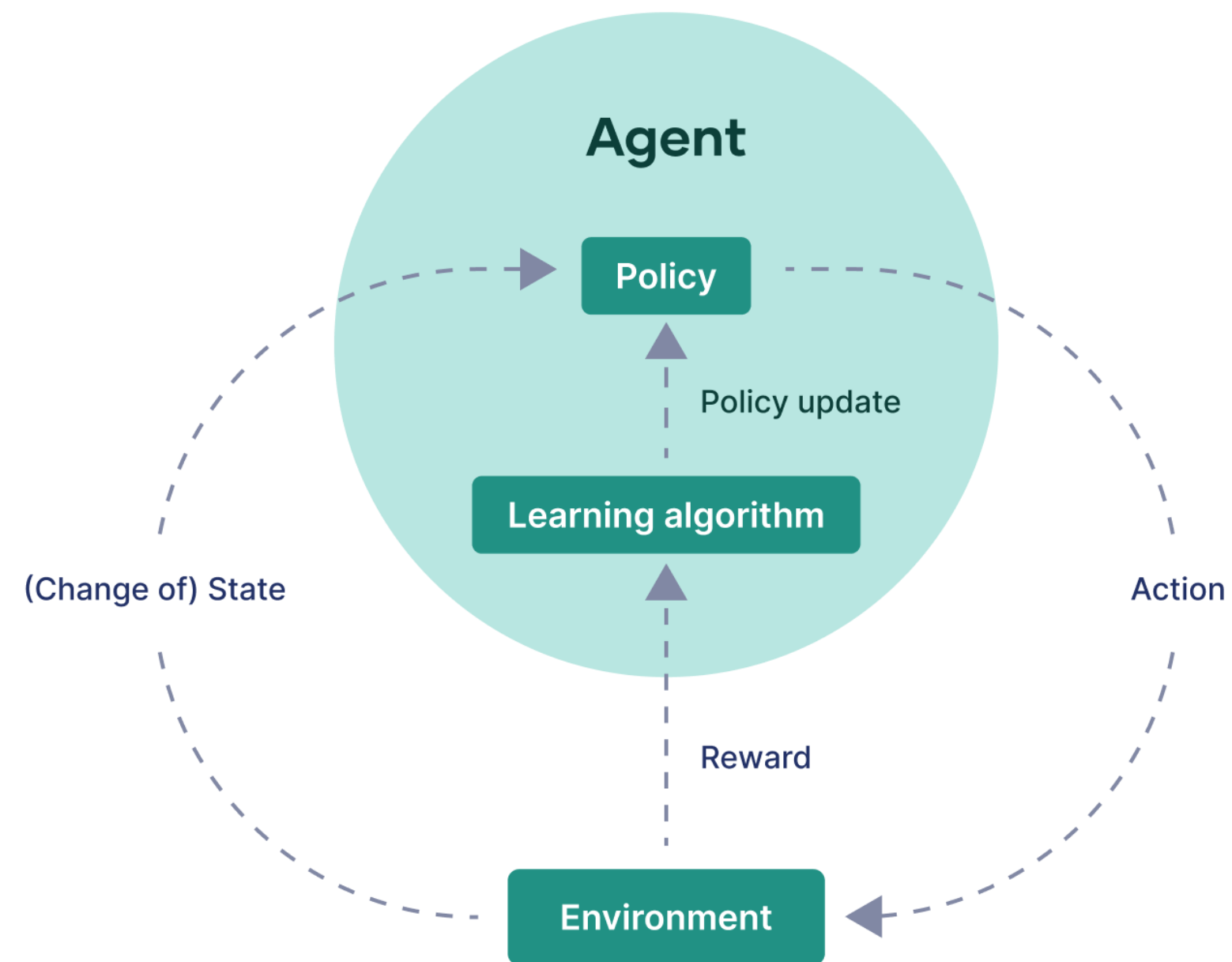
- Про RL в целом
- Offline RL для RecSys
 - Качество
 - Разнообразие
 - Long-term
- Online RL

Почему сегодня эта тема?

- RL – перспективное направление над разными разделами
- Предыдущие модели не решают ряд проблем, но RL может их решить

Про RL

The general framework of reinforcement learning



Про RL



Описание Таблица лидеров

Участвовать

Deep Reinforcement Learning

Изучаются алгоритмы обучения с подкреплением, использующие нейронные сети.

neuralnetworks

reinforcement learning

🤖 29 🌀 27 📌 27 🔥 25 😄 14 💪 12 🥰 11 😊 11 🤔 10 🕒 6 📺 5 🧙 4 🇺🇦 3 🥰 3 📄 3 👍 2 😓 2 😞 2 🧑 2 🧑 2 🐛 1 🍌 1 😂 1 +

Лектор

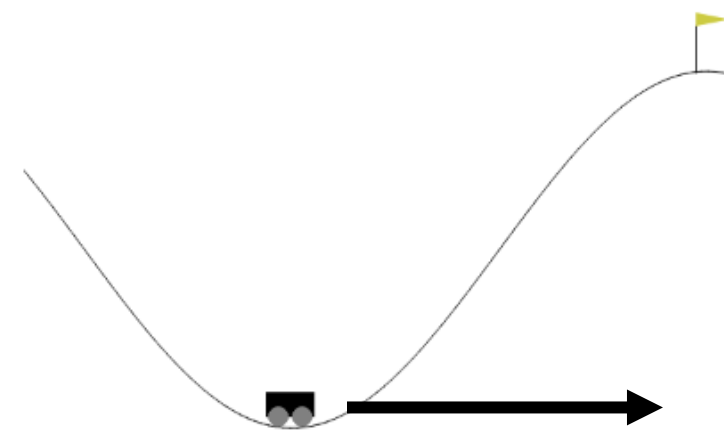
Антон Плаксин, исследователь в группе Yandex.Research и доцент Уральского федерального университета.

Автор курса



Anton Plaksin

Mountain Car



Action Space

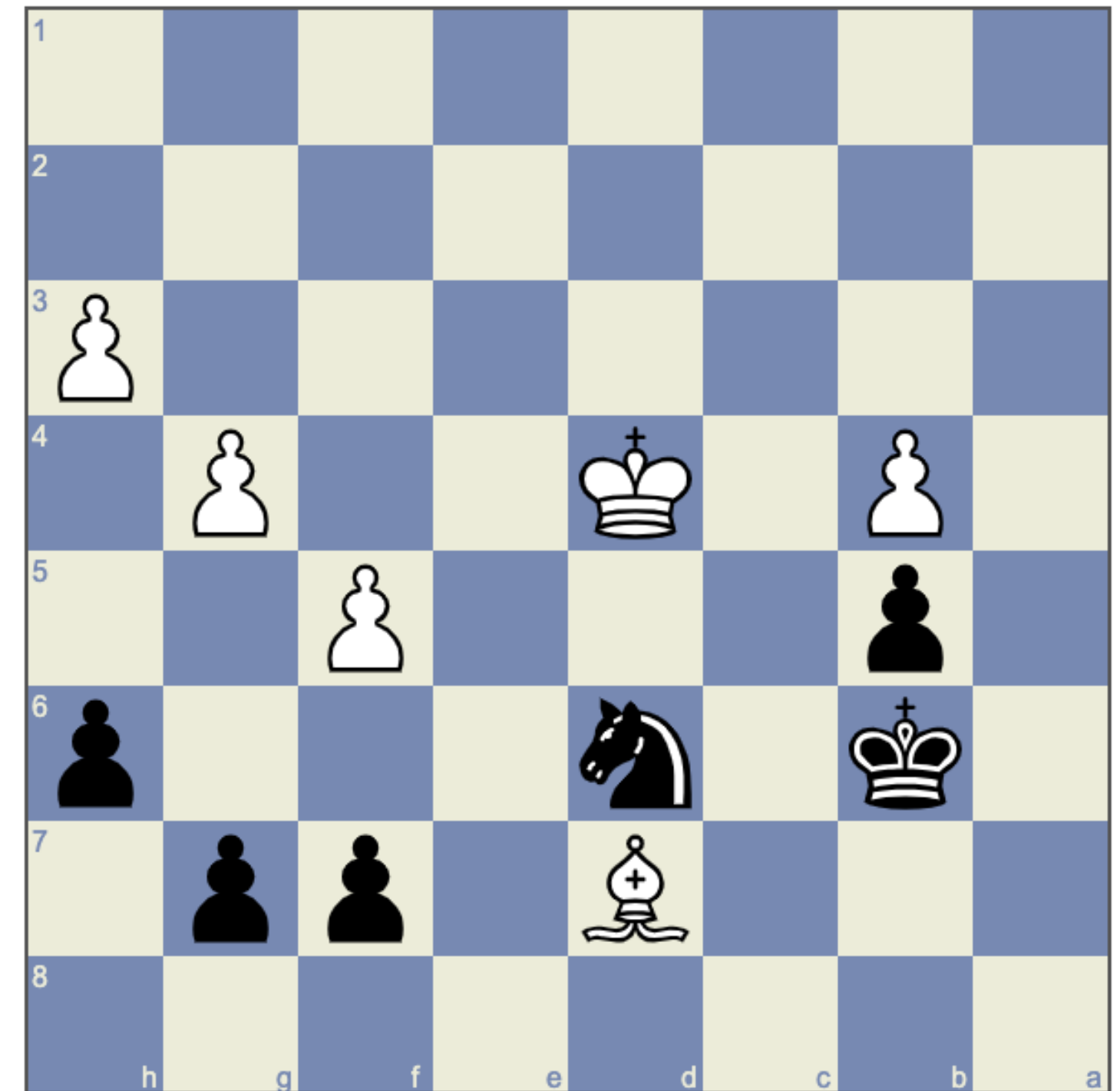
There are 3 discrete deterministic actions:

- 0: Accelerate to the left
- 1: Don't accelerate
- 2: Accelerate to the right

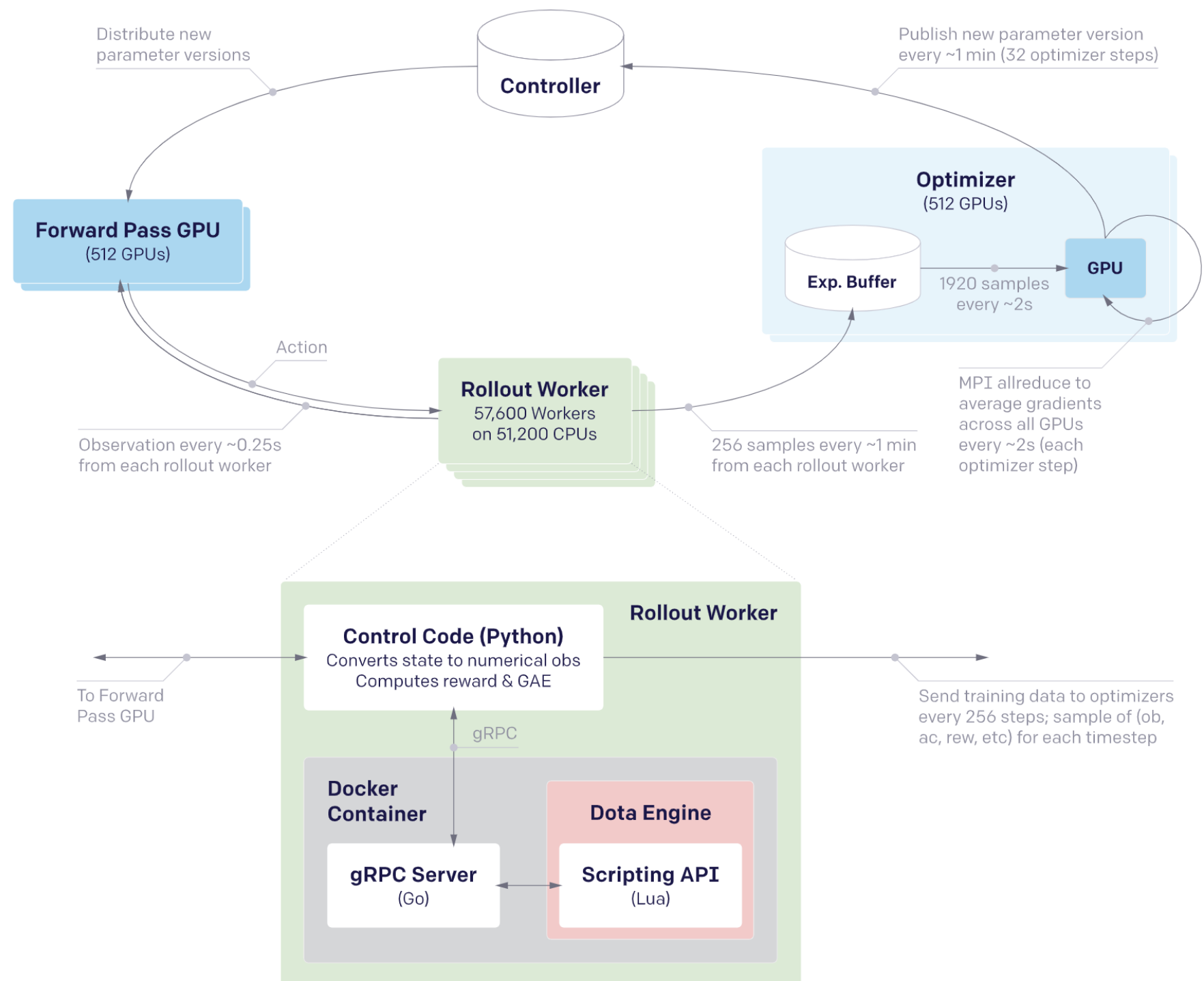
Action Space	<code>Discrete(3)</code>
Observation Space	<code>Box([-1.2 -0.07], [0.6 0.07], (2,), float32)</code>
import	<code>gymnasium.make("MountainCar-v0")</code>

Про RL

Жадная стратегия vs
долгосрочная оптимизация



Про RL



Dota 2 with Large Scale Deep Reinforcement Learning

OpenAI, *

Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław "Psyho" Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique Pondé de Oliveira Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, Susan Zhang

December 13, 2019



Про RL

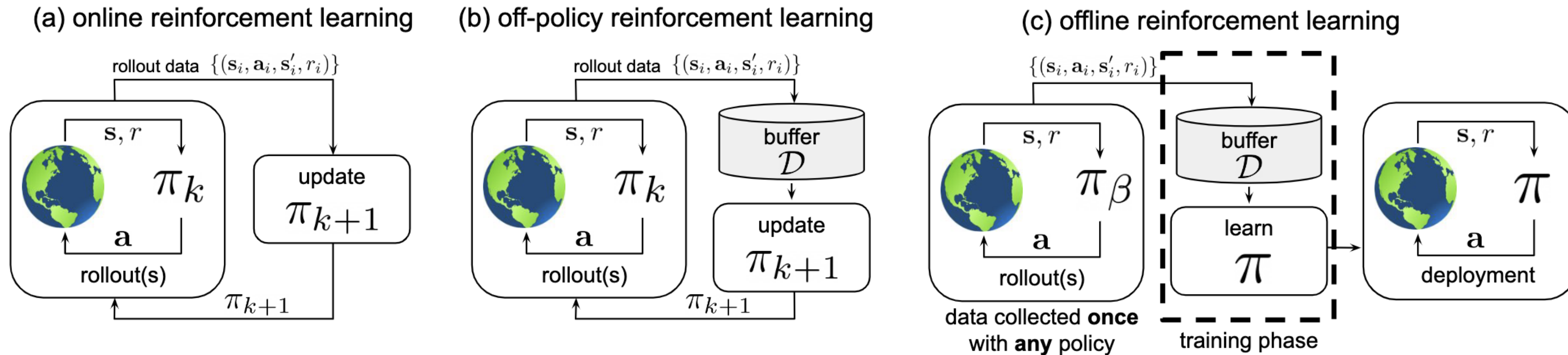
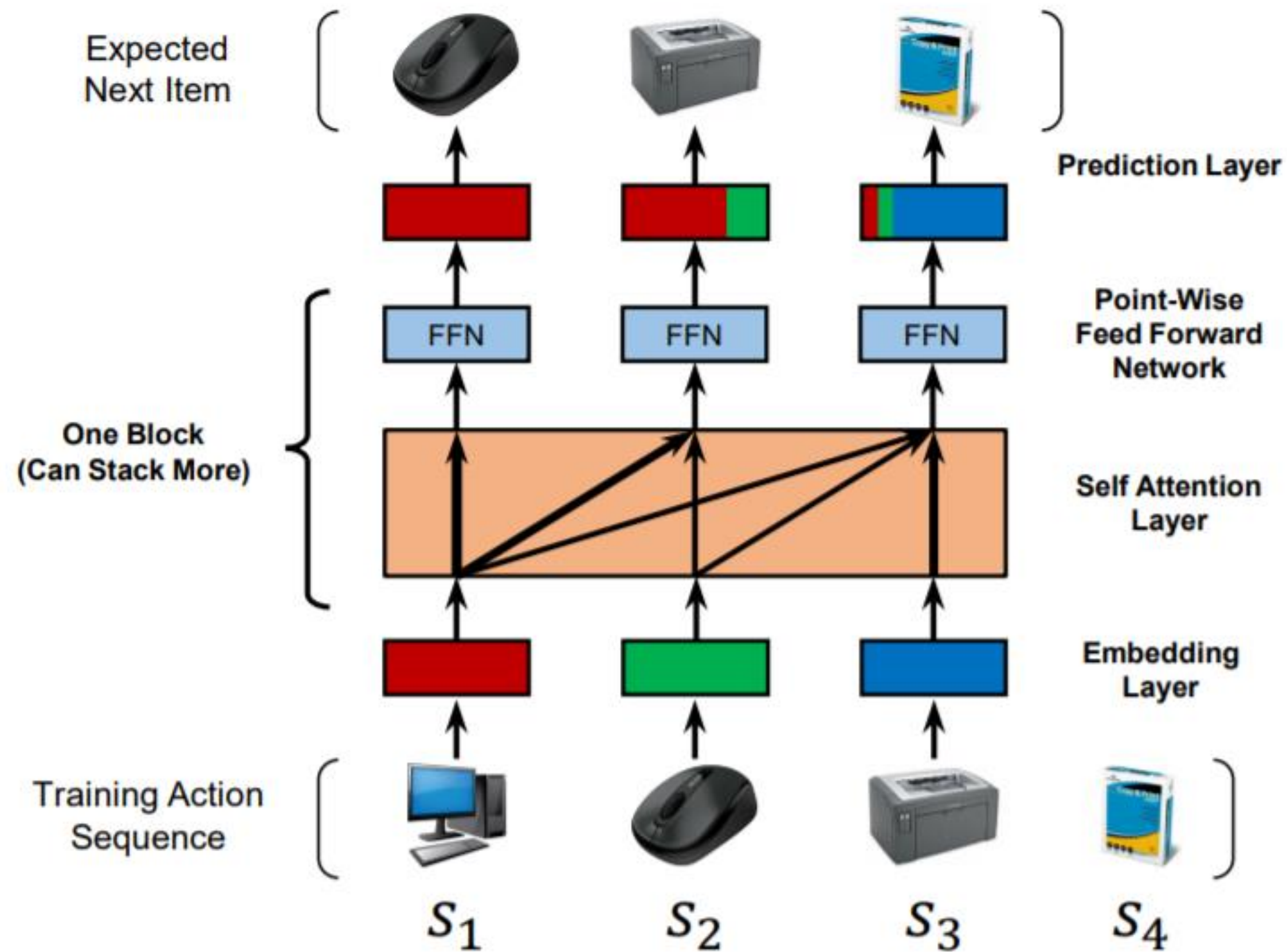


Figure 1: Pictorial illustration of classic online reinforcement learning (a), classic off-policy reinforcement learning (b), and offline reinforcement learning (c). In online reinforcement learning (a), the policy π_k is updated with streaming data collected by π_k itself. In the classic off-policy setting (b), the agent's experience is appended to a data buffer (also called a replay buffer) $\mathcal{D}$, and each new policy π_k collects additional data, such that $\mathcal{D}$ is composed of samples from $\pi_0, \pi_1, \dots, \pi_k$, and all of this data is used to train an updated new policy π_{k+1} . In contrast, offline reinforcement learning employs a dataset $\mathcal{D}$ collected by some (potentially unknown) behavior policy π_β . The dataset is collected once, and is not altered during training, which makes it feasible to use large previous collected datasets. The training process does not interact with the MDP at all, and the policy is only deployed after being fully trained.

RL & RecSys



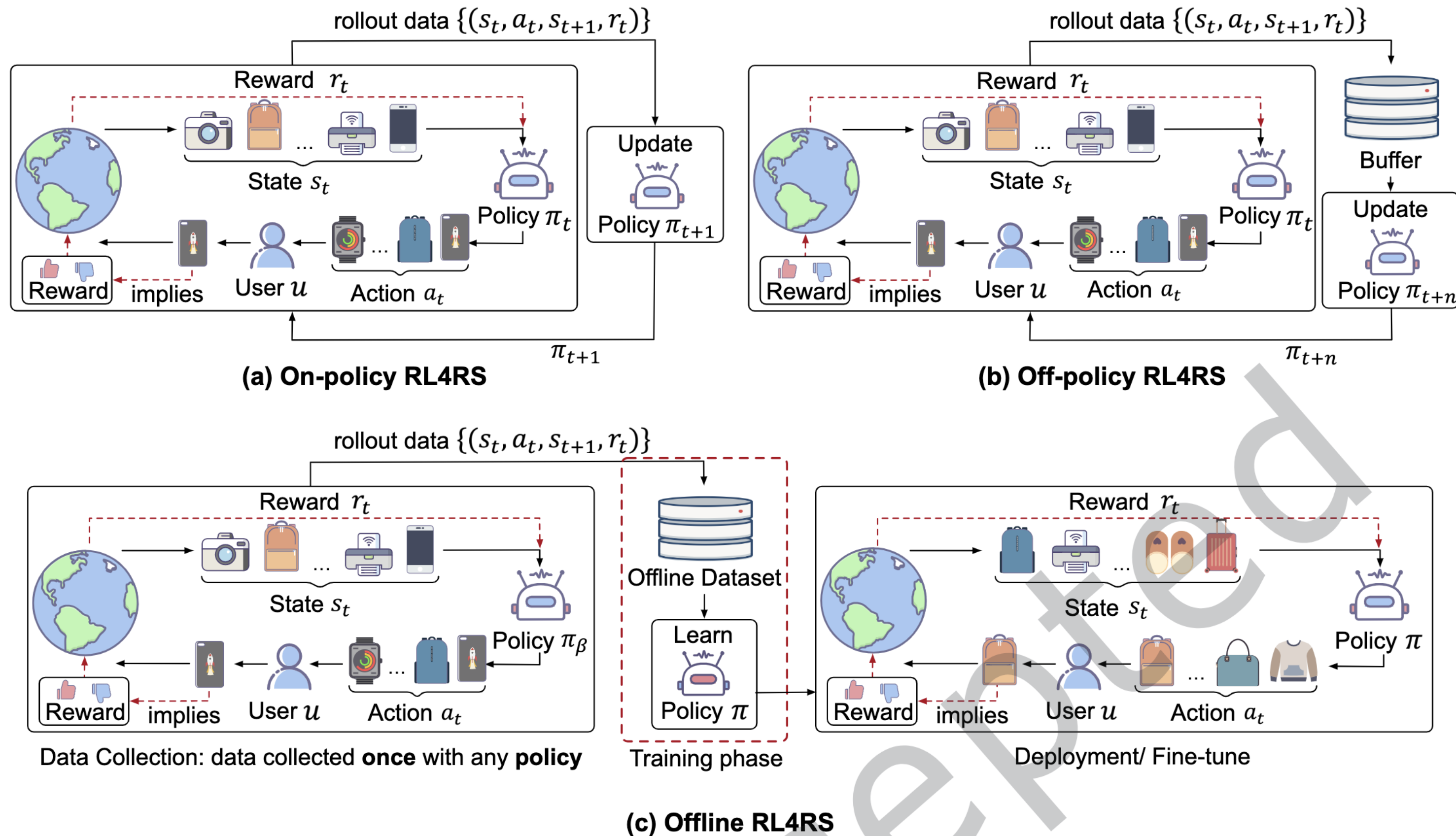
Что с того, что мы предсказываем следующий айтем?

RL & RecSys

- Бизнес хочет зарабатывать деньги, а не предсказывать next-item
- Выгодно растить ретеншн клиентов, развивать у них привычку пользоваться сервисом
- Всегда ли выгодно рекомендовать самый релевантный айтем?



RL & RecSys



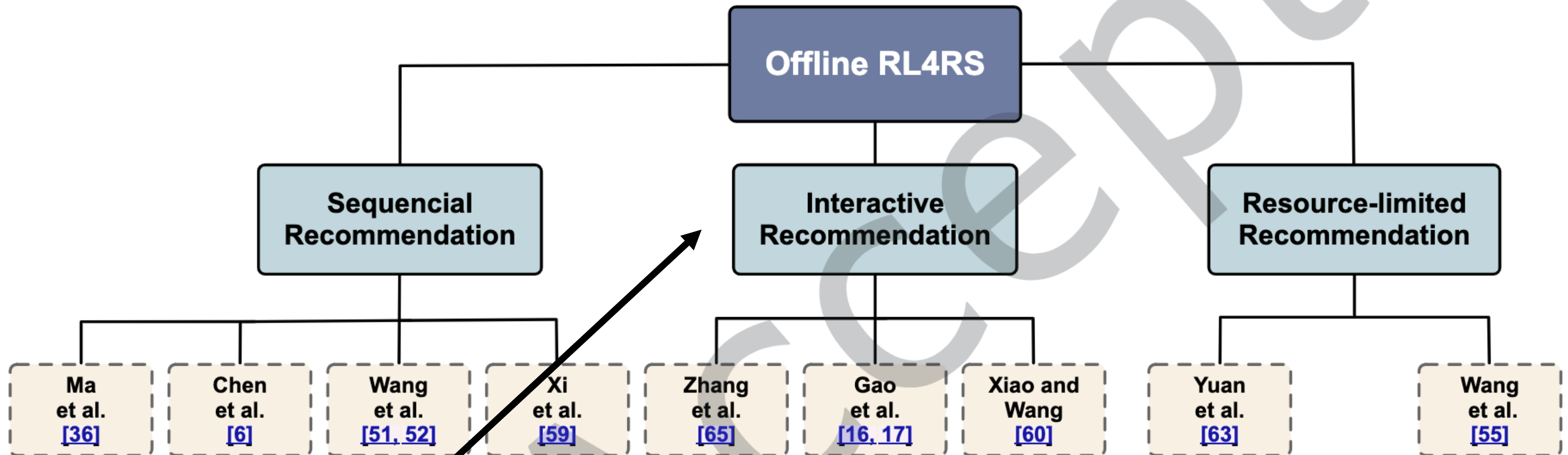


Fig. 4. Offline RL4RS Works Classifications

Можно думать как бин. поиск по интересам. Почему?



RL & RecSys



Что такое политика?

Политика (π) — это стратегия, определяющая **какие рекомендации выдавать** на основе:

- **Состояния (state):** Контекст пользователя (история, профиль, окружение).
- **Действия (action):** Выбор айтема для рекомендации.

Типы политик

1. **Детерминированная:** Четкое правило (например, «рекомендовать топ-5 по предсказанному CTR»).
2. **Стохастическая:** Вероятностный выбор (например, Thompson Sampling).
3. **Оптимальная:** Находится через RL (Q-Learning, Policy Gradients).

Примеры в рекомендациях

- **ϵ -Greedy:** С вероятностью ϵ исследуем случайные айтемы.
- **LinUCB:** Выбирает айтемы с максимальной верхней доверительной границей.
- **Deep Policy Network:** Нейросеть, предсказывающая вероятность выбора айтема.

В логах залогирована некоторая существующая политика!

RL & RecSys

1. Q-function: Оценка долгосрочной выгоды

В RL агент учится принимать решения, максимизируя **награду (reward)**.

Q-function (функция качества) отвечает на вопрос:

«Какую ожидаемую суммарную награду получит агент, если в состоянии s выберет действие a , а затем будет действовать оптимально?»

Формально:

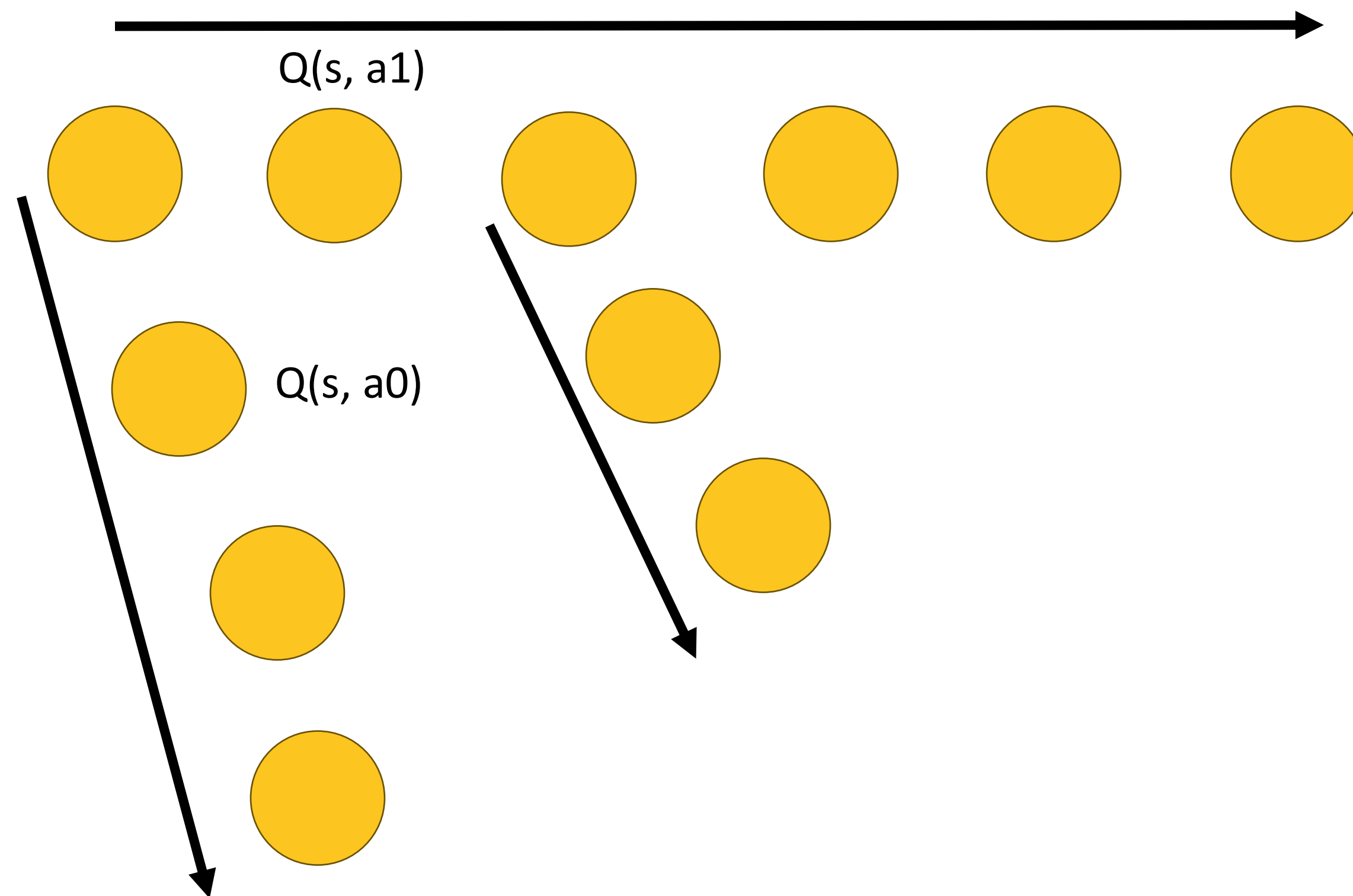
$$Q(s, a) = \mathbb{E} [R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots \mid s_t = s, a_t = a]$$

- γ — коэффициент дисконтирования (учитывает важность будущих наград).
- Чем выше $Q(s, a)$, тем лучше действие a в состоянии s .

Пример в RecSys:

- s — история взаимодействий пользователя,
- a — рекомендация (например, фильм),
- $Q(s, a)$ — оценка, как сильно рекомендация увеличит engagement.

RL & RecSys



RL & RecSys про качество

Supervised Advantage Actor-Critic for Recommender Systems

Xin Xin

School of Computer Science, Shandong University, China
xinxin@sdu.edu.cn

Ioannis Arapakis

Telefonica Research, Barcelona, Spain
ioannis.arapakis@telefonica.com

Alexandros Karatzoglou

Google Research, London, UK
alexkz@google.com

Joemon M. Jose

School of Computing Science, University of Glasgow, UK
Joemon.Jose@glasgow.ac.uk

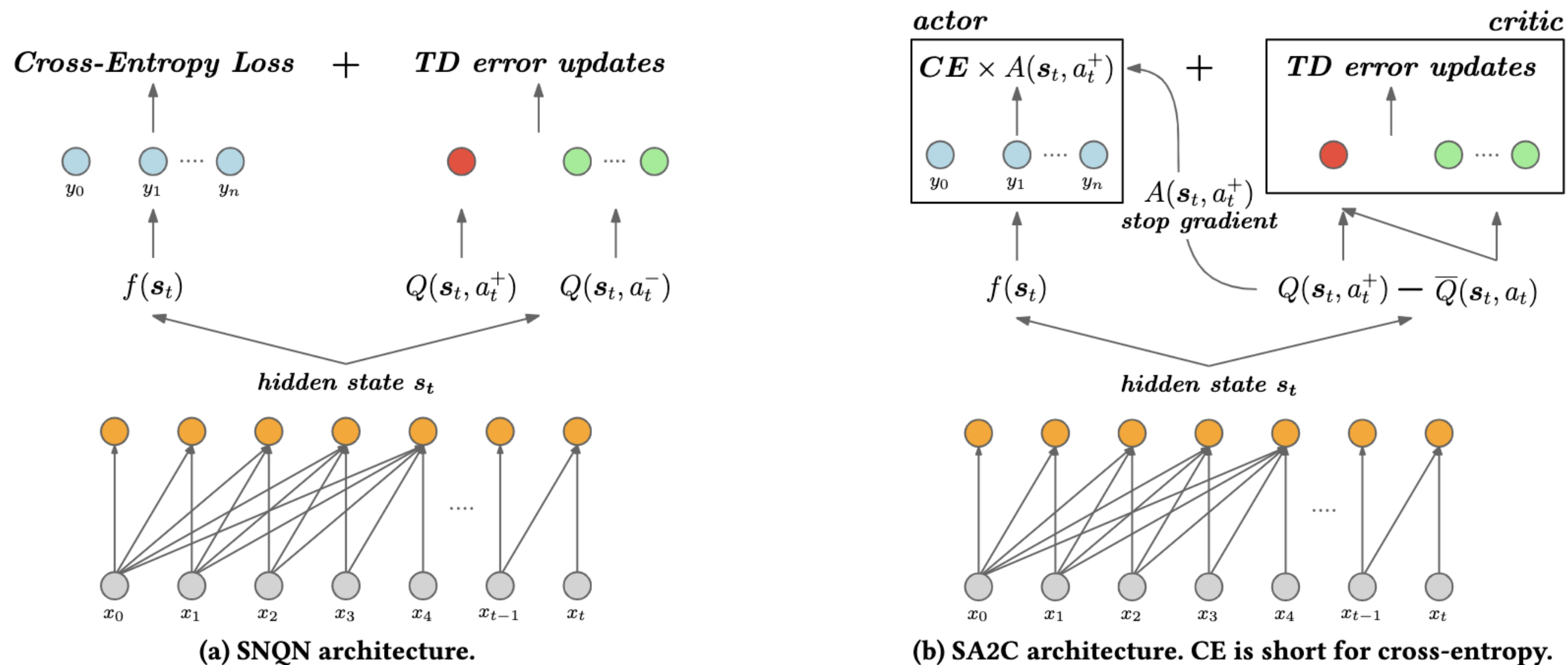


Figure 1: The learning framework architectures of SNQN and SA2C.

RL & RecSys

- $\mathcal{S}$: a continuous state space to describe the user state. The state of a user at timestamp t can be represented as $\mathbf{s}_t = G(\mathbf{x}_{1:t}) \in \mathcal{S}$ ($t > 0$).
- $\mathcal{A}$: a discrete action space which contains candidate items. The action a of the agent is to recommend the selected item. In off-line training data, we can get the positive action at timestamp t from the input sequence (i.e., $a_t^+ = x_{t+1}$ ($t \geq 0$)).
- $\mathbf{P}$: $\mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the state transition probability. When learning from off-line data, we can make an assumption that only positive actions can affect the user state. In other words, taking a negative (unobserved) action doesn't update the user state [12, 40].
- R : $\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, where $r(\mathbf{s}, a)$ denotes the immediate reward by taking action a at state $\mathbf{s}$. The flexible reward scheme allows the agent to optimize the recommendation models towards expectations that are not captured by simple supervised loss functions.
- ρ_0 is the initial state distribution with $\mathbf{s}_0 \sim \rho_0$.
- γ is the discount factor for future rewards.

$$\max_{\pi_{\theta}} \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)], \text{ where } R(\tau) = \sum_{t=0}^{|\tau|} \gamma^t r(\mathbf{s}_t, a_t),$$

$$L_q = \underbrace{(r(\mathbf{s}_t, a_t^+) + \gamma \max_{a'} Q(\mathbf{s}_{t+1}, a') - Q(\mathbf{s}_t, a_t^+))^2}_{L_p: \text{positive TD error}}$$

$$+ \underbrace{\sum_{a_t^- \in N_t} (r(\mathbf{s}_t, a_t^-) + \gamma \max_{a'} Q(\mathbf{s}_t, a') - Q(\mathbf{s}_t, a_t^-))^2}_{L_n: \text{negative TD error}},$$

Algorithm 1 Training procedure of SNQN

Input: user-item interaction sequence set $\mathcal{X}$, recommendation model $G(\cdot)$, reinforcement head $Q(\cdot)$, supervised head $f(\cdot)$, pre-defined reward function $r(s, a)$

Output: all parameters in the learning space Θ

- 1: Initialize all trainable parameters
- 2: Create $G'(\cdot)$ and $Q'(\cdot)$ as copies of $G(\cdot)$ and $Q(\cdot)$, respectively
- 3: **repeat**
- 4: Draw a mini-batch of $(x_{1:t}, a_t^+)$ from $\mathcal{X}$
- 5: Draw negative actions set N_t for $x_{1:t}$
- 6: $\mathbf{s}_t = G(x_{1:t}), \mathbf{s}'_t = G'(x_{1:t})$
- 7: $\mathbf{s}_{t+1} = G(x_{1:t+1}), \mathbf{s}'_{t+1} = G'(x_{1:t+1})$
- 8: Generate random variable $z \in (0, 1)$ uniformly
- 9: **if** $z \leq 0.5$ **then**
- 10: $a_*^+ = \arg\max_a Q(\mathbf{s}_{t+1}, a), a_*^- = \arg\max_a Q(\mathbf{s}_t, a)$
- 11: $L_p = (r(\mathbf{s}_t, a_t^+) + \gamma Q'(\mathbf{s}'_{t+1}, a_*^+) - Q(\mathbf{s}_t, a_t^+))^2$
- 12: $L_n = \sum_{a_t^- \in N_t} (r(\mathbf{s}_t, a_t^-) + \gamma Q'(\mathbf{s}'_t, a_*^-) - Q(\mathbf{s}_t, a_t^-))^2$
- 13: Calculate L_s and $L_{snqn} = L_s + L_p + L_n$
- 14: Perform updates by $\nabla_{\Theta} L_{snqn}$
- 15: **else**
- 16: $a_*^+ = \arg\max_a Q'(\mathbf{s}_{t+1}, a), a_*^- = \arg\max_a Q'(\mathbf{s}_t, a)$
- 17: $L_p = (r(\mathbf{s}_t, a_t^+) + \gamma Q(\mathbf{s}_{t+1}, a_*^+) - Q'(\mathbf{s}'_t, a_t^+))^2$
- 18: $L_n = \sum_{a_t^- \in N_t} (r(\mathbf{s}_t, a_t^-) + \gamma Q(\mathbf{s}_t, a_*^-) - Q'(\mathbf{s}'_t, a_t^-))^2$
- 19: Calculate L_s and $L_{snqn} = L_s + L_p + L_n$
- 20: Perform updates by $\nabla_{\Theta} L_{snqn}$
- 21: **end if**
- 22: **until** converge
- 23: return all parameters in Θ

$$Q(\mathbf{s}_t, a_t) = \delta(\mathbf{s}_t \mathbf{h}_t^T + b) = \delta(G(x_{1:t}) \mathbf{h}_t^T + b),$$

$G(\cdot)$ - SASRec

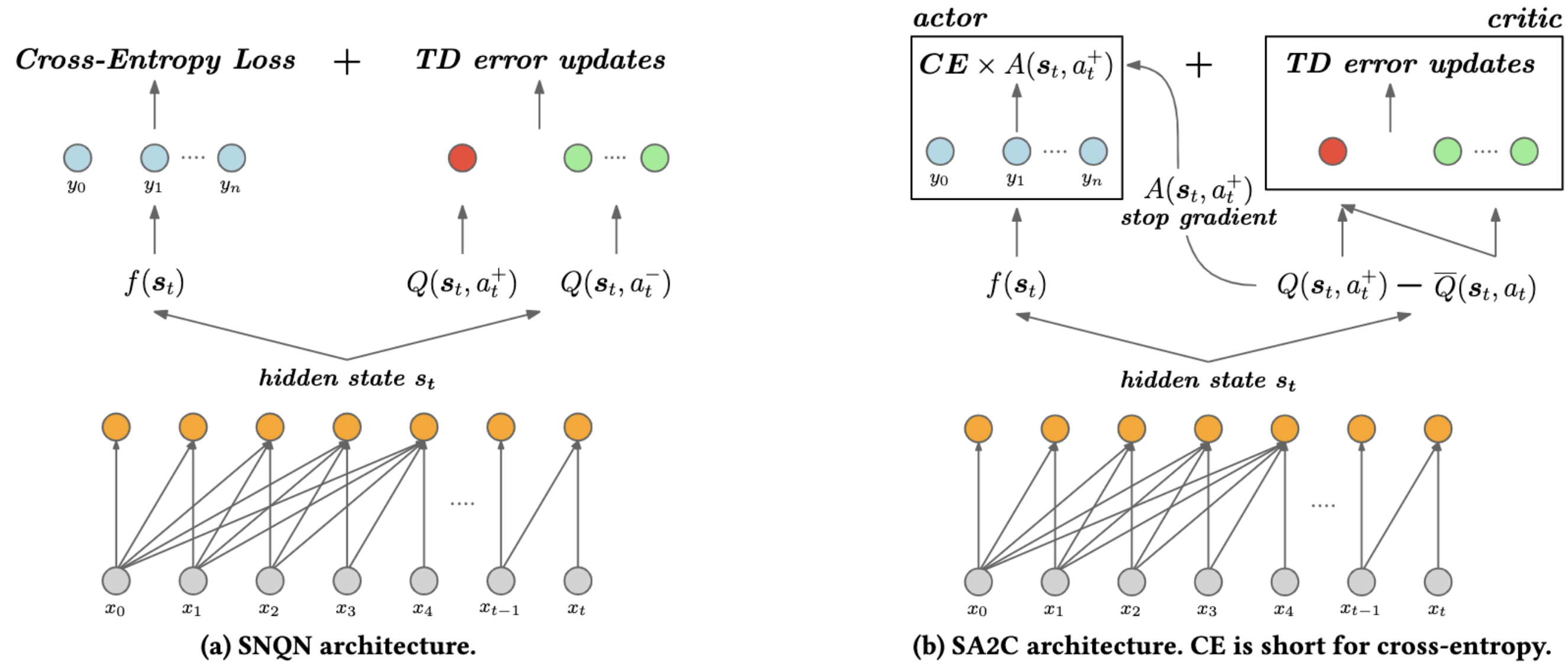


Figure 1: The learning framework architectures of SNQN and SA2C.

The advantage of an observed (positive) action is formulated as:

$$A(s_t, a_t^+) = Q(s_t, a_t^+) - \bar{Q}(s_t, a). \quad (7)$$

$$L_{sa2c} = L_a + L_q, \text{ where } L_a = L_s \cdot A(s_t, a_t^+).$$

RL & RecSys



Table 2: Top- k recommendation performance comparison of different models ($k = 5, 10, 20$) on RC15 dataset. Recommendations are generated from the supervised head. NG is short for NDCG. Boldface denotes the highest score.

Models	purchase						click					
	HR@5	NG@5	HR@10	NG@10	HR@20	NG@20	HR@5	NG@5	HR@10	NG@10	HR@20	NG@20
GRU	0.3994	0.2824	0.5183	0.3204	0.6067	0.3429	0.2876	0.1982	0.3793	0.2279	0.4581	0.2478
GRU-SQN	0.4228	0.3016	0.5333	0.3376	0.6233	0.3605	0.3020	0.2093	0.3946	0.2394	0.4741	0.2587
GRU-SNQN	0.4368	0.3115	0.5428	0.3460	0.6316	0.3686	0.3124	0.2164	0.4067	0.2469	0.4856	0.2669
GRU-SAC	0.4394	0.3154	0.5525	0.3521	0.6378	0.3739	0.2863	0.1985	0.3764	0.2277	0.4541	0.2474
GRU-SA2C	0.4514	0.3297	0.5606	0.3652	0.6420	0.3859	0.3287	0.2307	0.4214	0.2606	0.5000	0.2806
Caser	0.4475	0.3211	0.5559	0.3565	0.6393	0.3775	0.2728	0.1896	0.3593	0.2177	0.4371	0.2372
Caser-SQN	0.4553	0.3302	0.5637	0.3653	0.6417	0.3862	0.2742	0.1909	0.3613	0.2192	0.4381	0.2386
Caser-SNQN	0.4781	0.3460	0.5876	0.3816	0.6657	0.4015	0.2800	0.1951	0.3682	0.2237	0.4465	0.2436
Caser-SAC	0.4866	0.3527	0.5914	0.3868	0.6689	0.4065	0.2726	0.1894	0.3580	0.2171	0.4340	0.2362
Caser-SA2C	0.4917	0.3635	0.6000	0.3989	0.6796	0.4192	0.2948	0.2068	0.3835	0.2356	0.4596	0.2549
NItNet	0.3632	0.2547	0.4716	0.2900	0.5558	0.3114	0.2950	0.2030	0.3885	0.2332	0.4684	0.2535
NItNet-SQN	0.3845	0.2736	0.4945	0.3094	0.5766	0.3302	0.3091	0.2137	0.4037	0.2442	0.4835	0.2645
NItNet-SNQN	0.3969	0.2803	0.5039	0.3152	0.5876	0.3363	0.3153	0.2176	0.4098	0.2482	0.4896	0.2686
NItNet-SAC	0.3914	0.2813	0.4964	0.3155	0.5763	0.3357	0.2977	0.2055	0.3906	0.2357	0.4693	0.2557
NItNet-SA2C	0.4382	0.3171	0.5403	0.3505	0.6259	0.3722	0.3410	0.2395	0.4348	0.2699	0.5113	0.2897
SASRec	0.4228	0.2938	0.5418	0.3326	0.6329	0.3558	0.3187	0.2200	0.4164	0.2515	0.4974	0.2720
SASRec-SQN	0.4336	0.3067	0.5505	0.3435	0.6442	0.3674	0.3272	0.2263	0.4255	0.2580	0.5066	0.2786
SASRec-SNQN	0.4435	0.3163	0.5581	0.3535	0.6450	0.3742	0.3284	0.2267	0.4271	0.2588	0.5083	0.2794
SASRec-SAC	0.4540	0.3246	0.5701	0.3623	0.6576	0.3846	0.3130	0.2161	0.4114	0.2480	0.4945	0.2691
SASRec-SA2C	0.4705	0.3385	0.5756	0.3728	0.6648	0.3956	0.3444	0.2407	0.4402	0.2719	0.5194	0.2920

RL & RecSys про разнообразие

Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning

Aleksandr V. Petrov
University of Glasgow
United Kingdom
a.petrov.1@research.gla.ac.uk

Craig Macdonald
University of Glasgow
United Kingdom
craig.macdonald@glasgow.ac.uk

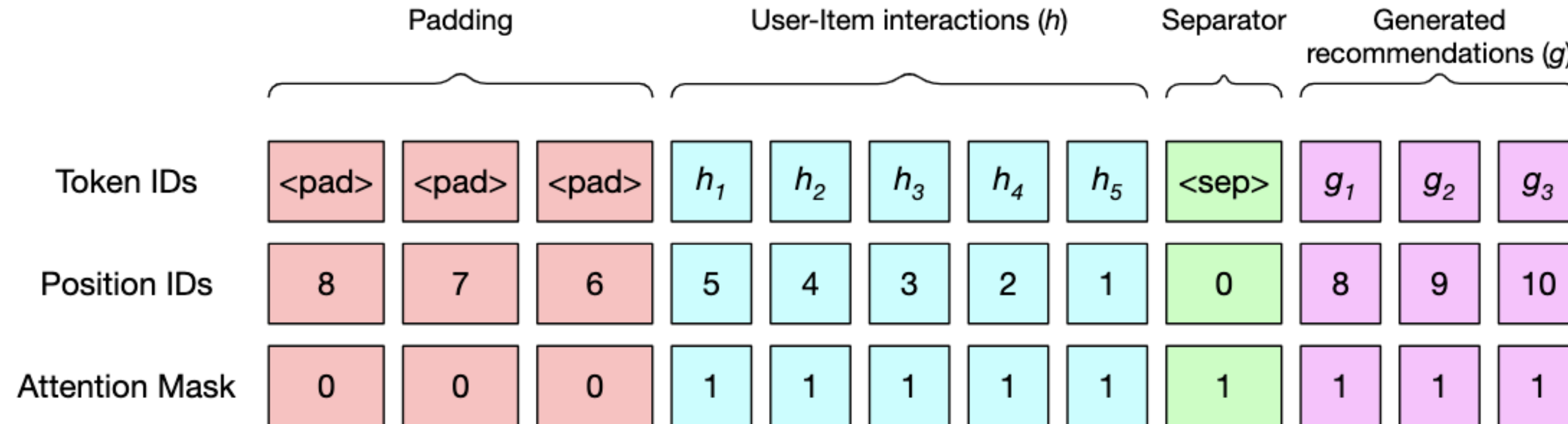


Figure 6: Proposed input sequence structure

RL & RecSys про разнообразие

Можно оптимизировать любую кастомную награду!

Model	Learning Objective
GPTRec-Reinforcement-NDCG	$\sum_{i=1}^K \frac{y_i}{\log(i+1)}$
GPTRec-Reinforcement-Diversity	$\sum_{i=1}^K \left[\frac{y_i}{\log(i+1)} + \lambda \cdot \frac{1}{K \cdot (K-1)} \sum_{j=1}^{i-1} \text{CosDist}(g_i, g_j) \right]$
GPTRec-Reinforcement-pCOUNT	$\sum_{i=1}^K \left[\frac{y_i}{\log(i+1)} - \lambda \cdot \text{freq}(g_i) \right]$

Table 3: Evaluation results. The best results are highlighted in bold; second best are underlined; * denotes significant difference with BERT4Rec (*pvalue* < 0.05, Bonferroni multi-test correction), arrows indicate the metric improvement direction

(a) MovieLens-1M

Model Type	Model	Secondary Metric	Secondary metric weight (λ)	Recall@1 $\uparrow$	Recall@10 $\uparrow$	NDCG@10 $\uparrow$	Diversity (ILD@10) $\uparrow$	Popularity Bias (nPCOUNT@10) $\downarrow$
Baselines	Popularity			0.0056 ^{<}	0.0363 ^{<}	0.0178 ^{<}	0.3222 ^{>}	1.0000 ^{>}
	MF-BPR			0.0113 ^{<}	0.0719 ^{<}	0.0366 ^{<}	0.2745	0.2867 ^{<}
	SASRec			0.0464 ^{<}	0.2482 ^{<}	0.1306 ^{<}	0.2703 ^{<}	<u>0.2389</u> ^{<}
	GPTRec-Shifting			0.0603	0.2647 ^{<}	0.1486 ^{<}	0.2749	0.2985 ^{<}
	BERT4Rec			0.0608	0.2921	0.1617	0.2746	0.3222
Supervised	GPTRec-Supervised (MC Teacher)			0.0791 ^{>}	0.2690 ^{<}	0.1638	0.2798 ^{>}	0.4470 ^{>}
NDCG Tuned	GPTRec-Reinforcement-NDCG			0.0829 ^{>}	<u>0.2785</u>	0.1682	0.2878 ^{>}	0.4205 ^{>}
Diversity tuned	GPTRec-Reinforcement-Diversity-0.2	ILD	0.2	0.0798 ^{>}	0.2298 ^{<}	0.1499 ^{<}	<u>0.3621</u> ^{>}	0.5212 ^{>}
	GPTRec-Reinforcement-Diversity-1.0	ILD	1.0	0.0795 ^{>}	0.1748 ^{<}	0.1272 ^{<}	0.4349 ^{>}	0.5016 ^{>}
Tuned to decrease popularity bias	GPTRec-Reinforcement-pCOUNT-3.0	PCOUNT	3.0	<u>0.0820</u> ^{>}	0.2733 ^{<}	<u>0.1669</u>	0.2774	0.3936 ^{>}
	GPTRec-Reinforcement-pCOUNT-6.0	PCOUNT	6.0	0.0710	0.2154 ^{<}	0.1392 ^{<}	0.2648 ^{<}	0.1742 ^{<}

RL & RecSys про long-term



User Retention-oriented Recommendation with Decision Transformer

Kesen Zhao
City University of Hong Kong
kesenzhao2-c@my.cityu.edu.hk

Lixin Zou[†]
Wuhan University
zoulixin15@gmail.com

Xiangyu Zhao[†]
City University of Hong Kong
xianzhao@cityu.edu.hk

Maolin Wang
City University of Hong Kong
MorinWang@foxmail.com

Dawei Yin
Baidu Inc.
yindawei@acm.org

tention). For user retention, we define it as the number of users logging in the platform over the next K time intervals (e.g., next K days, next K months)

$$e_t = \sum_{k=1}^K \mathbb{1}[\text{log-in}_{t+k}], \quad (2)$$

where $\mathbb{1}$ is the indicator function, log-in_{t+k} is a binary function that indicates whether the user logs in the platform or not at the next $(t + k)$ -th timestep.

RL & RecSys про long-term

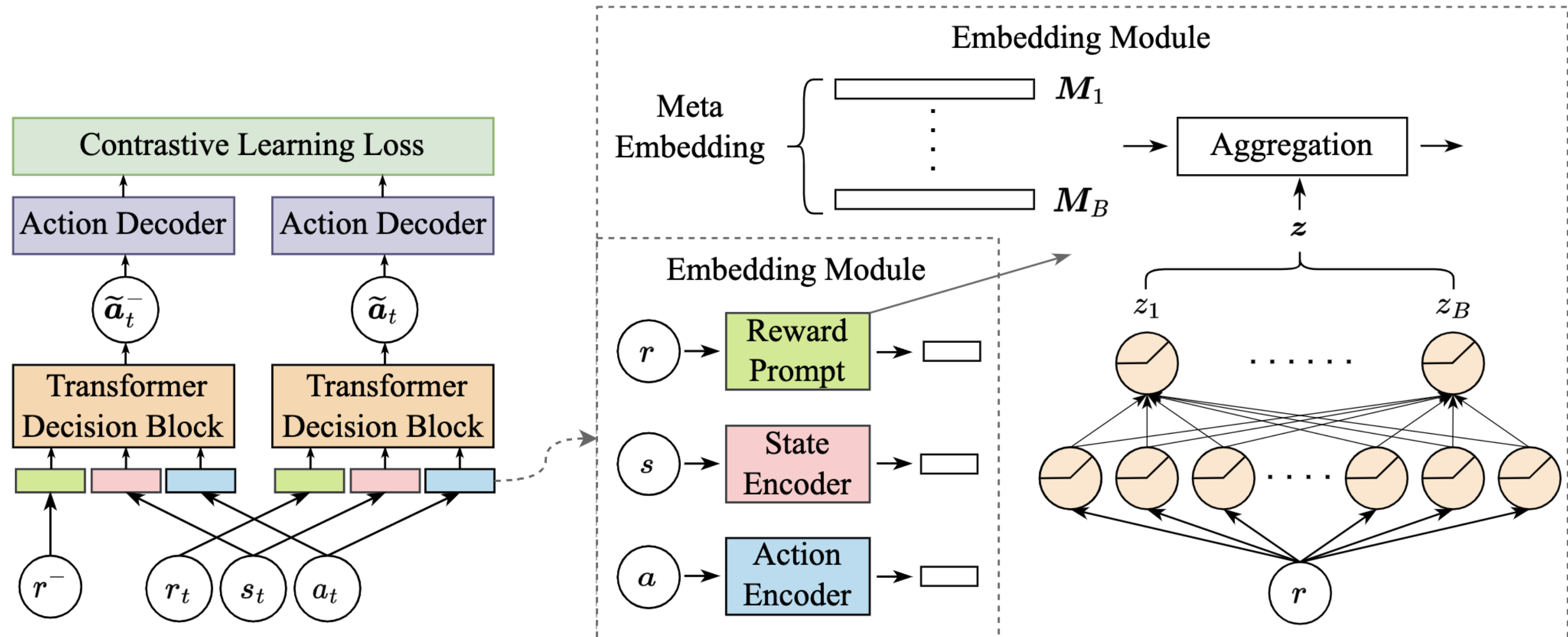


Figure 1: Framework overview of DT4Rec. Negative samples have the same states and actions as positive samples, while rewards are replaced with different values. Positive and negative samples share the same model parameters.

RL & RecSys про long-term

2.7.1 Weighted Contrastive Loss. For each sample, we use same state and action, with different rewards, as negative samples, denoting its predicted action embedding as $\tilde{V}^-$. The weighted contrastive learning loss can be formulated as

$$\mathcal{L}_{CL} = - \sum_{\tilde{V}^- \in \Upsilon} \kappa(\tilde{V}^-) f_s(\tilde{V}, \tilde{V}^-), \quad (13)$$

where Υ is the set of negative samples. $f_s(\cdot)$ calculates the similarity of two sequences by averaging of embeddings' dot product at every row of the matrices $\tilde{V}$ and $\tilde{V}^-$. $\kappa(\tilde{V}^-)$ is the weighting hyperparameter set according to the reward value of negative sample, i.e., the weight is inversely proportional to reward. The reason is that smaller rewards make user retention lower and we want to be dissimilar to those ones.

Besides the weighted contrastive loss, we also optimize the DT with the original cross-entropy loss as

$$\hat{Y} = \psi(W_v \tilde{V} + b_v) \quad (14)$$

$$\mathcal{L}_{CE} = CE(\hat{Y}, Y), \quad (15)$$

where Y is the label matrix with the one-hot label in every row, $\hat{Y}$ is the corresponding predicted label distribution. Here, CE is the cross-entropy loss function, ψ is the softmax function. W_v and b_v are learnable parameters.

Finally, the overall loss function is formulated by combining the cross-entropy loss and weighted contrastive loss as

$$\mathcal{L} = \mathcal{L}_{CE} + \beta \mathcal{L}_{CL}, \quad (16)$$

where β is a hyper-parameters.

RL & RecSys про long-term

- **SB-URS.** The **similarity-based user return score** evaluates the effectiveness of recommended item sequences by weighted sum over the ground truth user retention score. Specifically, we divide the samples into 8 classes according to their ground truth reward and calculate BLEU-1 scores between predicted sequences and ground truth for each class as their similarity. Then, the SB-URS are calculated as follows:

$$\text{SB-URS} = \sum_{k=0}^K s_k \cdot \left(g_k - \frac{K}{2} \right) \cdot N_k, \quad (17)$$

where s_k is the similarity of class k . g_k is the ground truth reward of class k . We want the similarity to be as small as possible for samples with small ground truth reward, and as large as possible for samples with large ground truth reward. N_k is the number of samples with a reward of k .

- **Improved user retention (IUR).** It measures the percentage of relative users' average retention improvement compared to their logged average retention in the offline data, which directly reflects users' engagement on the system. And a larger percentage of IUR indicates that recommendation lists make more users engaged in the system.
- **No return count (NRC).** It is the percentage of users who left the system after a recommendation was made. A smaller no return count indicates that recommendation lists keep more users engaged in the system.

Table 3: Overall performance comparison in user retention. The best performance is marked in bold font.

Dataset	Model	Metric			
		MB-URS↑	SB-URS↑	IUR↑	NRC↓
IQiYi	DT4Rec-R	5.16	52131	7.5%	2.6%
	TopK	5.41	61045	12.7%	2.0%
	LIRD	5.63	63572	17.3%	1.9%
	DT4Rec	6.05*	72270*	26.0%*	1.4%*
ML-1m	DT4Rec-R	5.42	13050	31.2%	9.7%
	TopK	5.71	13964	38.3%	6.9%
	LIRD	5.86	14627	41.9%	4.8%
	DT4Rec	5.93*	15562*	43.6%*	3.6%*

“*” indicates the statistically significant improvements (i.e., two-sided t-test with $p < 0.05$) over the best baseline.
 ↑: the higher the better; ↓: the lower the better.

RL & RecSys про long-term

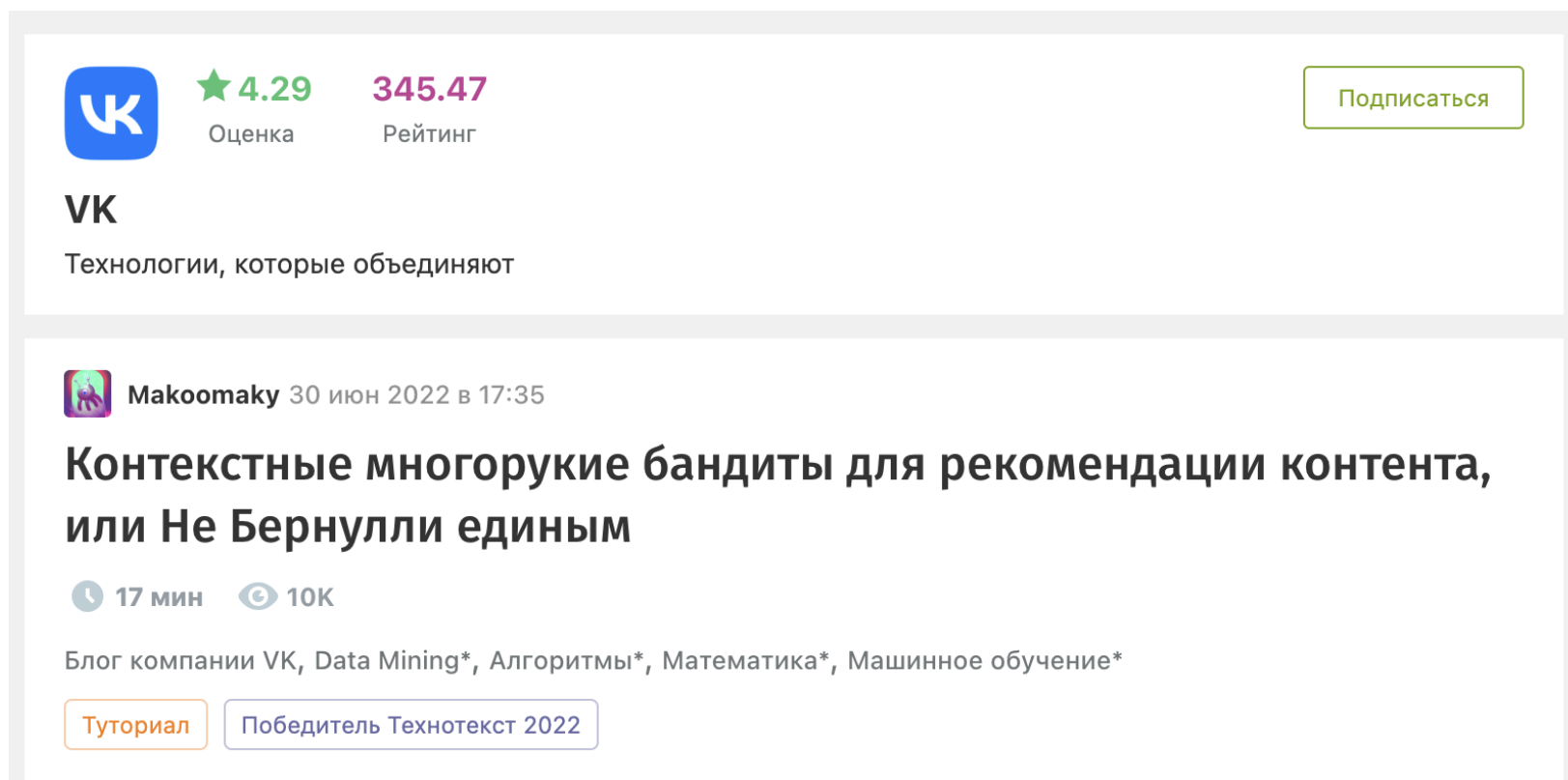
Table 2: Overall performance comparison in prediction accuracy. The best performance is marked in bold font.

Dataset	Model	Metric			
		BLEU↑	ROUGE↑	NDCG↑	HR↑
IQiYi	BERT4Rec	0.7964	0.7693	0.7384	0.7673
	SASRec	0.8009	0.7906	0.7827	0.7815
	DT4Rec	0.8249*	0.8172*	0.8139*	0.8044*
ML-1m	BERT4Rec	0.3817	0.3806	0.5286	0.2769
	SASRec	0.4052	0.3983	0.5409	0.3123
	DT4Rec	0.4331*	0.4185*	0.5679*	0.3342*

“*” indicates the statistically significant improvements (i.e., two-sided t-test with $p < 0.05$) over the best baseline.

↑: the higher the better; ↓: the lower the better.

Многорукие бандиты



- у нас есть история взаимодействия пользователя $u_s \in U$ и контента $i_r \in I$;
- исходя из задач бизнеса, некоторые из видов взаимодействия считаются положительными, а некоторые отрицательными. То есть каждой паре взаимодействия (u_s, i_r) мы ставим в соответствие ответ $y_{sr} \in Y$, а также опционально вес w_{sr} характеризующий значимость данного примера. Здесь стоит отметить, что парой взаимодействия могут стать пользователь и единица контента, которые на самом деле никак не связаны. Такая необходимость появляется, когда не хватает отрицательных сигналов;
- каждой паре (u_s, i_r) ставятся в соответствие признаки $x_{sr} \in X$ это могут быть как описания пользователя или контента, так и просто их id (например, в случае коллаборативной фильтрации);
- наша задача — на основе X и Y научить модель понимать, у каких новых пар пользователей и единиц контента в будущем есть шанс положительно провзаимодействовать.

Многорукие бандиты

- Алгоритмы учатся на истории, на формирование которой сами же и повлияли. Однако этот факт нигде не учитывается и может приводить к нежелательным смещениям в обучении.
- Усложняется поиск ниши для потенциально качественного, но самобытного контента.
- В рекомендациях в основном показывается контент на темы, которые когда-то интересовали пользователя, а новые темы платформа не предлагает.
- Непонятно, что делать с перманентным появлением на платформе нового контента, который потенциально может быть качественным и достойным большой аудитории. Крайним случаем является запуск новой платформы рекомендаций или появление на сайте нового типа контента. Тогда в нашем распоряжении вообще нет никакой истории взаимодействия, на которой можно учиться.

Многорукие бандиты

- игрок приходит в казино со слот-машинами (также известными как однорукие бандиты) и, естественно, хочет выиграть как можно больше;
- слот-машина с некоторой вероятностью выдаёт фиксированный выигрыш на каждую попытку игрока дёрнуть за ручку автомата, но у каждого однорукого бандита своя вероятность выигрыша;
- достоверно известно, что есть однорукий бандит с вероятностью получения выигрыша, ради которой есть смысл его искать;
- таким образом, цель игрока — как можно быстрее найти наиболее выгодного однорукого бандита и играть в него.



Многорукие бандиты

Эпсилон-жадный подход

Самым простым подходом является эпсилон-жадный алгоритм. В общем виде он работает следующим образом:

- мы с вероятностью ε выбираем ручку бандита, у которой на основе уже полученных данных наибольший средний выигрыш, а с вероятностью $1 - \varepsilon$ — случайную ручку;
- обновляем математические ожидания выигрыша у всех ручек;
- повторяем с первого пункта.

К задаче бернуллиевских бандитов этот подход применяется тривиальным образом:

- для каждой ручки считаем отношение количества выигрышей к количеству попыток;
- с вероятностью ε пробуем ручку с максимальным отношением — это exploitation, а с вероятностью $1 - \varepsilon$ пробуем случайную ручку — и реализуем exploration;
- получая новые данные, обновляем отношения.

Недостаток этого подхода состоит в нерациональном exploration. По мере поступления данных нам становится понятно, что какие-то ручки более перспективны для исследования, чем другие. Однако они всё равно получают одинаковые шансы быть рекомендованными пользователю на очередной итерации exploration.

Многорукие бандиты

Для дальнейшего рассмотрения принципов работы многоруких бандитов нам нужен способ обучения моделей, который позволит:

1. В потоковом режиме (то есть без сохранения обучающей выборки в памяти и постоянного её переиспользования) комбинировать новые данные с уже имеющимися у нас представлениями о среде, получая новую, более точную модель среды.
2. Получать не просто точечные оценки параметров модели, но и их вероятностное распределение, которое ещё и помогает понять, насколько мы уверены в текущих оценках.

Многорукие бандиты

В общем виде сэмплирование Томпсона работает следующим образом:

- из текущего апостериорного распределения параметров среды θ сэмплируем их значения;
- считая результаты сэмплирования за истинные значения параметров среды, выбираем действие, при котором математическое ожидание выигрыша наибольшее;
- получив новые данные, пересчитываем обновлённое апостериорное распределение параметров θ , считая предыдущее апостериорное распределение за априорное;
- повторяем всё с первого пункта.

Многоручкие бандиты

Применительно к задаче бернуллиевских бандитов этот алгоритм выглядит следующим образом. Среда описывается набором параметров θ_k (по одному на каждую k -ую ручку), каждый такой параметр имеет априорное бета-распределение. На каждой итерации мы:

- из каждого бета-распределения параметров θ_k наших ручек сэмплируем их значение $\Theta_k \sim p(\theta_k)$;
- просэмплированные значения Θ_k считаем за вероятность той или иной ручки принести выигрыш, таким образом, оптимальным действием является $\arg \max_k \Theta_k$;
- получив новую информацию от среды в ответ на наши действия, пересчитываем значения бета-распределения параметров по схеме, описанной [выше](#);
- повторяем всё с первого пункта.

Многорукие бандиты



По прошествии какого-то времени мы собрали следующие данные: первая набрала 2 000 лайков и 100 000 просмотров, вторая ни одного лайка и всего 10 просмотров, третья — 1 лайк и 10 000 просмотров. Лайки мы здесь считаем за те самые выигрыши, а просмотры за попытки.

На стадии exploitation будет выбран первый кот — как наиболее перспективный на основе текущей информации.

Многорукие бандиты

- либо первая картинка, у которой $\alpha = 2000 + 1 = 2001$ (считаем, что априорное значение $\alpha = 1$), $\beta = 100000 - 2000 + 1 = 98001$ (считаем, что априорное значение $\beta = 1$), а следовательно, математическое ожидание равно

$$\frac{\alpha}{\alpha + \beta} = \frac{2001}{2001 + 98001} \approx 0,02$$

(по формуле мат. ожидания бета-распределения). Такое математическое ожидание располагает наше бета-распределение фотографии первого кота правее всех остальных бета-распределений.

- либо вторая фотография, у которой $\alpha = 0 + 1 = 1$, $\beta = 10 - 0 + 1 = 11$, а следовательно, дисперсия будет равна

$$\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)} = \frac{11}{144 \cdot 13} \approx 0,006.$$

При такой относительно большой дисперсии вероятность появления больших значений Θ_i велика.

Третья же картинка будет иметь весьма небольшие шансы быть выбранной, так как у неё и

небольшое математическое ожидание $\frac{2}{2 + 10000} \approx 0,0002$,

и небольшая дисперсия $\frac{2 * 10000}{(2 + 10000)^2(2 + 10000 + 1)} \approx 0,00000002$.

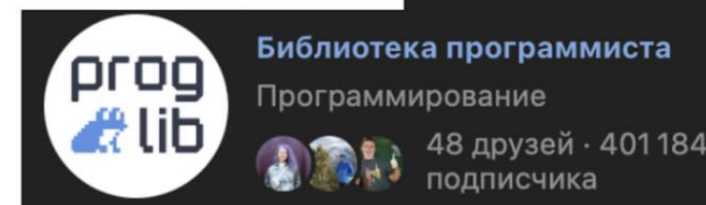
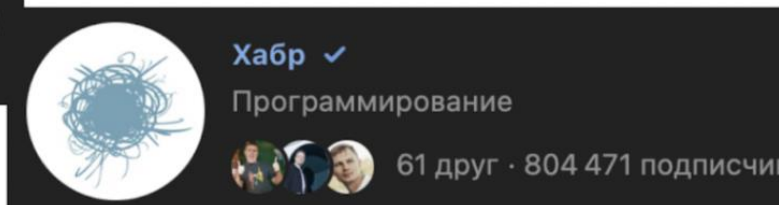
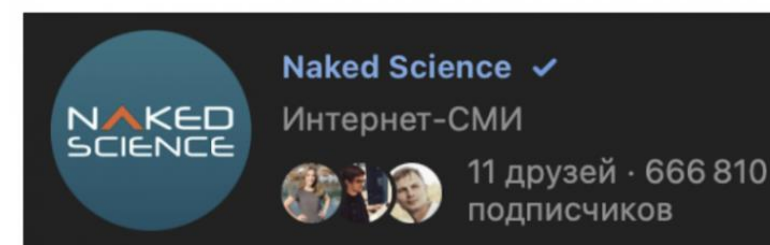
Многоручкие бандиты

Логистическая регрессия + вариационный вывод

Пришло время глобально поменять модель среды. Если раньше вероятность бинарного выигрыша при использовании k -ой ручки зависела от одного параметра θ_k , то сейчас мы вводим для каждой ручки вектор параметров $\theta_k = (\theta_k^1, \dots, \theta_k^v)$. Пусть контекст характеризуется вектором признаков $x = (x^1, \dots, x^v)$, тогда вероятность выиграть при использовании k -ой ручки будет равна

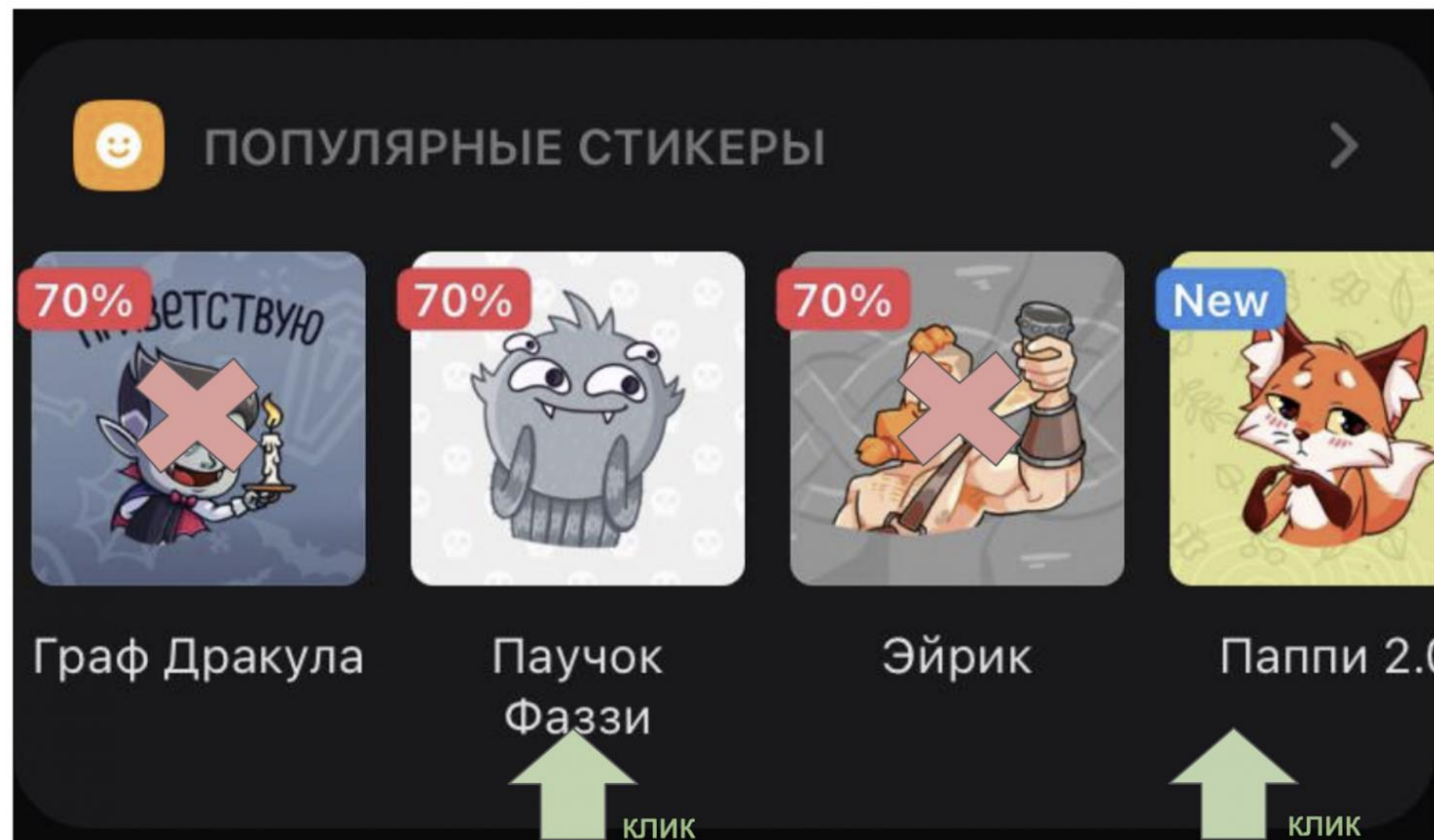
$$\sigma(\theta_k^T \cdot x) = \frac{1}{1 + e^{-\theta_k^T \cdot x}},$$

где сигмоида гарантирует, что значение, которое мы получаем от модели будет между 0 и 1.



Многорукие бандиты

Мы решили считать, что взаимодействие носило отрицательный характер в том случае, когда пользователь точно увидел рекомендуемый объект и при этом сознательно не проявил к нему интереса. Для этого в качестве примеров отрицательного взаимодействия мы берём все объекты в горизонтальных скролл-виджетах, которые расположены слева от объекта, нажатого пользователем. На рисунке ниже такими примерами отрицательных взаимодействий являются «Граф Дракула» и «Эйрик».



Сначала метрики в тестовой группе незначительно упали относительно контроля, но через две недели начали уверенно расти. Через два месяца эффект составил +8% установок игр, +2% запусков, после чего мы раскатали новые рекомендации на всю аудиторию. Результаты на стикерах против классических ML-рекомендаций дали +5% к выручке от их продажи.

Про симуляции

➤ Collection: 88 papers in RecSys conferences (2020 – 2022)

No. papers	Percentage	Train/test split	Global timeline?
30	34%	Random split	No
22	25%	Leave-one-out	No
17	19.5%	Single time point	Partially
15	17%	Simulation-based online	Yes
4	4.5%	Sliding window	Yes



Почему симулятор необходим даже в случае построения идеальной модели отклика пользователя на айтем внутри симулятора?

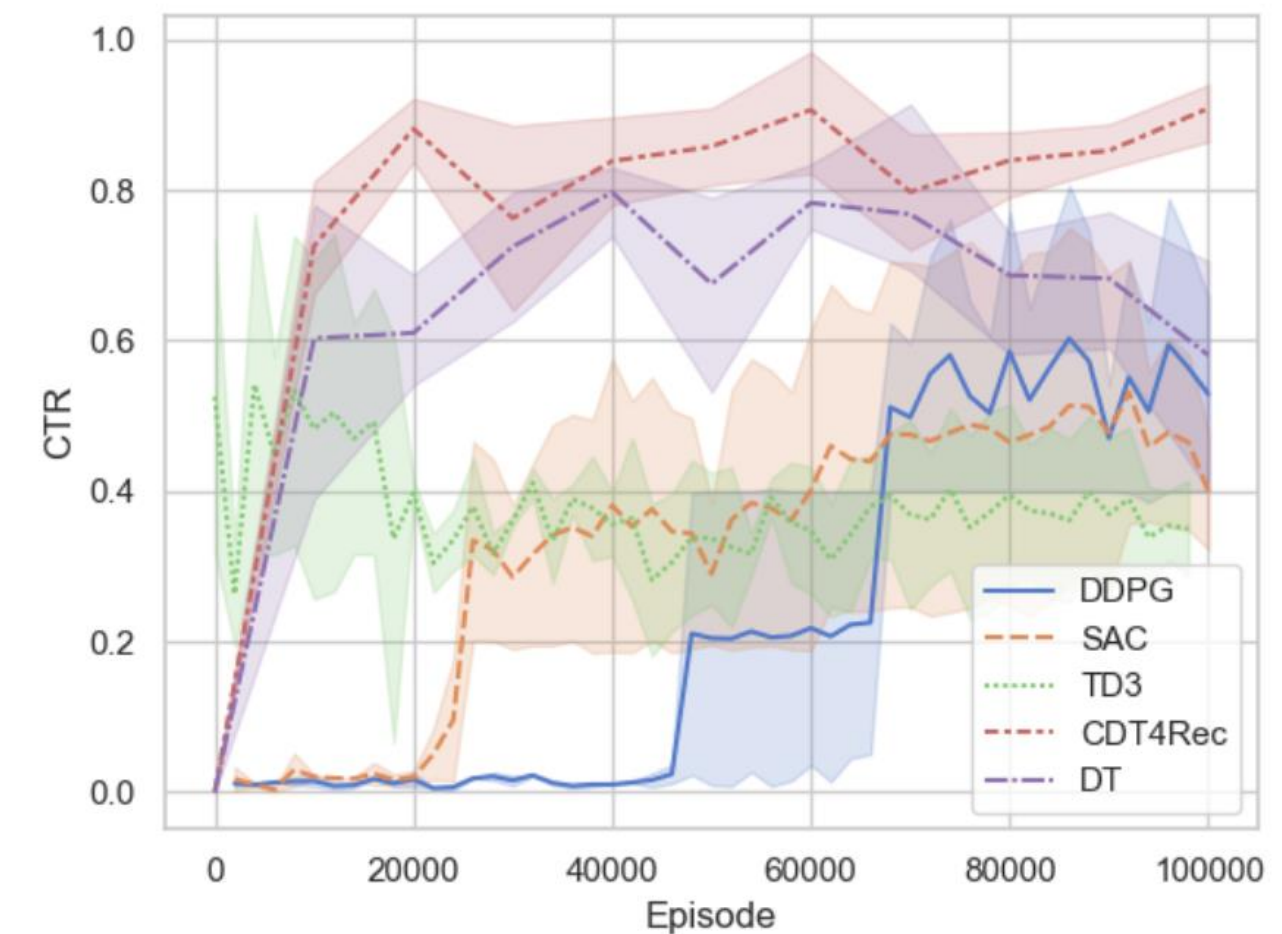


Figure 2: Overall comparison result with variance between the baselines and CDT4Rec in the VirtualTaobao simulation environment.

Главная проблема – плохое качество симуляции

Про реальный пример из практики



fotol 14 фев в 13:50

RL RecSys в проде: хайп или игра вдолгую

Средний

8 мин

1.8K

Блог компании Т-Банк, Машинное обучение*

Кейс

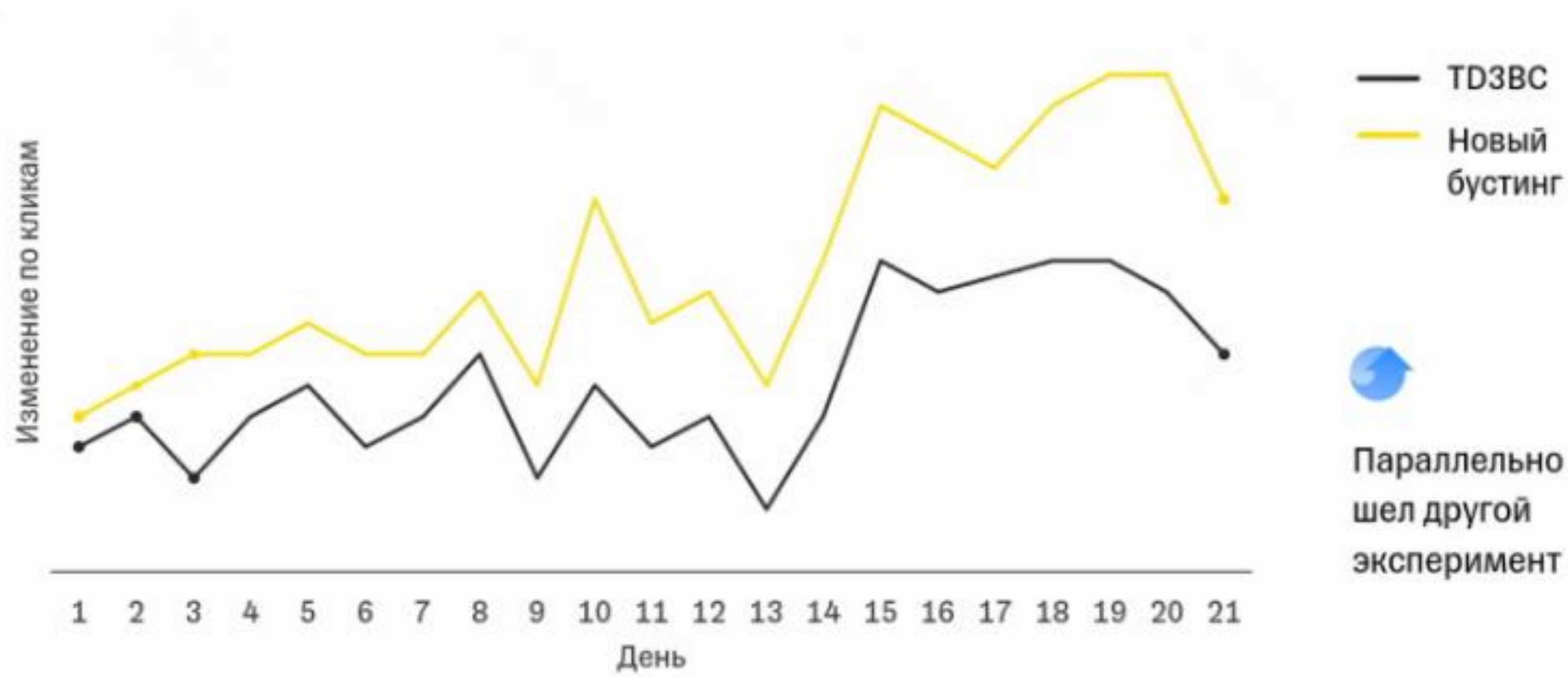


График АБ-теста по количеству переходов на оффер для двух разных групп

Про реальный пример из практики

1

2

3

4

Actions

User -vector

Сюда не вставить:

1) Конверсию в клик по айтому

2) Сколько раз юзер кликал на action

Потому что user и action (items) разделены

Поэтому RL проиграло бустингу, в котором user-item/user-category фичи насчитаны

Заключение

- Многорукие бандиты – онлайн, но можно обучить в оффлайне, не оптимизируют долгосрок
- Сложный RL скорее оффлайн, full online рискованно
- Бандиты – exploration/exploitation, RL – любая награда
- Offline eval – ограничивает RL оптимизацию

Спасибо за
внимание